

A continuación, se presenta una reorganización explicativa de los componentes técnicos esenciales utilizados para la gestión y conexión segura de bases de datos:

1. PHP Data Objects (PDO) En versiones anteriores de PHP, la comunicación con las bases de datos se realizaba mediante funciones tradicionales como `mysqli_query`, las cuales presentaban importantes vulnerabilidades de seguridad. En la actualidad, PDO se erige como una clase moderna y orientada a objetos que opera como un traductor universal entre el código y el motor de datos. Su principal beneficio radica en la protección automática que brinda contra ataques de "SQL Injection" (uno de los métodos de hackeo más extendidos a nivel global) a través de la implementación de "Sentencias Preparadas". Cuando se ejecuta la instrucción `$this->pdo = new PDO(...)`, se está inicializando este traductor y suministrándole las credenciales de acceso a MySQL.
2. Data Source Name (DSN) El DSN (`$dsn`) consiste en una cadena de texto (String) que establece los parámetros de localización o coordenadas esenciales para PDO. Su función es especificar las directrices de conexión, tales como el uso de una base de datos MySQL, el host correspondiente, el nombre de la base y el tipo de codificación del texto.
3. La Constante Mágica **DIR** En el ecosistema de PHP, **DIR** es identificada como una "constante mágica". Cuenta con la propiedad de reconocer con total exactitud la carpeta en la que se encuentra almacenado el archivo en ejecución, lo que facilita la localización de rutas relativas de manera independiente a la computadora donde se implemente el sistema.
4. Gestión de Errores: `try { ... } catch { ... }` y `PDOException` Para garantizar la estabilidad de la aplicación, se estructuran los siguientes mecanismos de contingencia: Estructura Try-Catch: Funciona como una red de seguridad operativa. La instrucción determina que el sistema intente (`try`) procesar un bloque de código; en caso de que ocurra una falla crítica, el sistema captura (`catch`) la anomalía para evitar el despliegue de una pantalla de error incomprensible para el usuario, permitiendo mostrar un mensaje amigable en su lugar. `PDOException`: Representa la alarma específica del sistema de datos. Si PDO no logra establecer el enlace debido a que MySQL se encuentra inactivo o a que los datos de acceso son erróneos, se genera e interrumpe el flujo mediante una alerta denominada `PDOException`.

Archivo: `modelos/Conexion.php` Su propósito fundamental es establecer un enlace técnico entre la lógica del sistema y el motor de MySQL bajo estrictos protocolos de integridad.

En cuanto a la infraestructura de comunicación, se emplea PDO (PHP Data Objects). Esta herramienta se desempeña como un mediador universal que resguarda la aplicación de forma nativa frente a vectores de ataque por inyección SQL. Para optimizar los recursos del servidor, se implementa el Patrón Singleton (`private static $instancia`). Mediante la privacidad del método `__construct()`, se impide la generación de conexiones redundantes. La lógica se centraliza en `getInstance()`, función encargada de verificar la existencia previa del enlace para su reutilización o creación exclusiva. La estabilidad se gestiona mediante el bloque `try { ... } catch { ... }`, que opera como una red de seguridad operativa. Este mecanismo permite que, ante una falla crítica de MySQL, el sistema capture (`catch`) la anomalía y despliegue una respuesta controlada en lugar de interrumpir la experiencia del usuario con errores técnicos.

5. Archivo: `publico/test_bd.php` Consiste en un módulo de diagnóstico temporal diseñado para validar la integridad de las fases iniciales de configuración.

El procedimiento consistió en la integración de la clase `Conexion` mediante las instrucciones `require_once` y el espacio de nombres correspondiente. Tras solicitar la instancia de datos, se verificaba el estado de Apache y

MySQL en el entorno XAMPP a través de una confirmación visual en pantalla.

6. Archivo: ayudantes/Sesion.php Actúa como el guardián de seguridad del sistema, estableciendo una barrera perimetral para restringir el acceso a los módulos administrativos según el estado de autenticación.

Dada la naturaleza del protocolo HTTP, PHP utiliza la Superglobal `$_SESSION`. Este repositorio temporal en el servidor almacena la información vital del usuario, como su identidad y privilegios, tras una validación exitosa de credenciales. Debido a que las funciones se definieron como static, se emplea la referencia `self::` para la invocación de herramientas internas de la clase, eliminando la necesidad de instanciar nuevos objetos.

Controles de Ciberseguridad (OWASP): `session_regenerate_id(true)`: Actualiza el identificador de sesión durante el acceso para mitigar riesgos de Session Fixation. Control de Expiración (`time() + 1800`): Implementa una política de Timeboxing que finaliza la sesión tras 30 minutos de inactividad. Depuración de Cookies: Durante el cierre de sesión, se eliminan los registros locales en el navegador para garantizar la máxima protección de los datos.

7. Archivo: modelos/Usuario.php Su propósito fundamental es desempeñarse como el mediador técnico (Modelo de Datos) encargado exclusivamente de la comunicación con la tabla de usuarios en MySQL.

Bajo una Arquitectura Orientada a Objetos, se define una clase cuya responsabilidad única es la interacción con la información. Mediante el método `__construct()`, se inicializa el enlace al solicitar la instancia a la clase `Conexion`, garantizando que al instanciar `new Usuario()`, la conexión al motor de datos se establezca de forma automática. Para garantizar la Protección contra Inyección SQL, el método `buscarPorEmail` implementa el uso de "Marcadores" (`:email`). Esta técnica evita la concatenación directa de variables en la instrucción SQL, mitigando uno de los vectores de ataque más críticos en el desarrollo web. El flujo operativo de PDO (`prepare -> execute -> fetch`): `prepare()`: Transmite la estructura de la consulta a MySQL para su compilación segura y preventiva. `execute()`: Realiza la sustitución técnica del marcador por el valor real proporcionado por el usuario. `fetch(PDO::FETCH_ASSOC)`: Recupera la información obtenida y la transforma en un Arreglo Asociativo, facilitando su lectura mediante claves descriptivas como `$usuario['contrasena_hash']`.

8. Archivo: controladores/AuthControlador.php Objetivo: Recibir el formulario de login, procesarlo de forma segura y decidir a qué pantalla redirigir al usuario.

El patrón Controlador: Es el director de orquesta. Primero atrapo los datos que el usuario escribió (`$_POST`). Luego, llamo al Modelo de Datos (`buscarPorEmail`) para pedirle información a la base de datos. Finalmente, si todo está correcto, llamo a mi Clase de Seguridad (`Sesion::iniciarLogin`) y le digo al navegador hacia dónde ir (`header("Location: ...")`). `$_SERVER['REQUEST_METHOD']`: Lo usamos para distinguir si el usuario recién entró a la página por primera vez (método GET, entonces le mostramos el formulario), o si acaba de hacer clic en el botón "Ingresar" (método POST, entonces procesamos sus datos). Función `password_verify()`: Una genialidad de PHP. Como en la base de datos guardamos las contraseñas encriptadas con BCrypt (nunca se guardan en texto plano por seguridad), esta función compara mágicamente el texto que escribió el usuario (ej: "MiClave123") con el garabato inentendible guardado en la BD (ej: "\$2y\$10\$e8wYVjFw6rNKGZqfD7h..."). Si el algoritmo matemático coincide, devuelve true. Manejo de Errores Vía Interfaz: Si el email o la contraseña están mal, creamos una variable `$error = "..."` y cargamos nuevamente el HTML del login (`require`). El archivo HTML tomará esa variable y dibujará un cartelito rojo en la pantalla.

9. Archivo: publico/index.php (Front Controller) Objetivo: Ser el único punto de entrada de toda la aplicación por motivos de seguridad y ruteo centralizado.

Seguridad Front Controller: Evita que los usuarios naveguen por carpetas internas (como /modelos/ o /controladores/). Todo el mundo entra por publico/index.php. Carga Centralizada: Es el mejor lugar para importar la configuración global y las herramientas que usamos siempre (como la Base de Datos o la Sesión). Ruteo Dinámico (Routing): En lugar de tener 50 archivos distintos (login.php, registro.php, perfil.php), usamos variables \$_GET en la URL. Ejemplo: ?c=auth&a=mostrarLogin significa Controlador = AuthControlador, Acción = mostrarLogin(). Clases Variables: PHP tiene un superpoder que nos permite instanciar clases usando textos dinámicos. Primero armamos el texto con el nombre de la clase (\$claseCompleta = "App\\Controladores\\AuthControlador" 😊) y luego creamos el objeto escribiendo new \$claseCompleta();

10. Archivo: controladores/AdminControlador.php Su finalidad principal es salvaguardar el perímetro del área administrativa y organizar la información que se proyectará en el tablero de control (Dashboard).

Control de Acceso (__construct): Esta clase prioriza la ejecución de su constructor de forma inmediata. Al integrar Sesión::requerirLogin(), se establece una restricción técnica: si un usuario intenta vulnerar la ruta directa sin una autenticación previa, el sistema interrumpe el acceso y lo redirige al módulo de acceso. El método dashboard(): Representa el núcleo operativo del controlador. En fases posteriores, se encargará de realizar las consultas SQL para cuantificar usuarios y torneos vigentes, suministrando estos indicadores a la interfaz dashboard.php.

11. Concepto: Adaptación de Vistas y Rutas Absolutas (URL_BASE) Garantiza la correcta carga de activos digitales (estilos CSS, imágenes y scripts) con total independencia del controlador que invoque la interfaz visual.

Conflicto de Rutas Relativas (../): Bajo el patrón MVC, la interpretación del navegador se centraliza en publico/index.php, perdiendo la noción de la jerarquía física de carpetas que se utilizaba en maquetación estática. La solución estratégica: Se implementa la constante URL_BASE para establecer una referencia absoluta. Esto obliga al sistema a localizar los recursos desde el directorio raíz del proyecto de forma unificada. Arquitectura Limpia (CSS vs PHP): Se prohíbe el uso de estilos en línea para preservar la separación de responsabilidades. La lógica de presentación se confina a archivos .css, mientras que el .php se reserva exclusivamente para la lógica de datos. 12. Actualización: ayudantes/Sesion.php Optimización del repositorio temporal del servidor para almacenar metadatos adicionales tras una validación de identidad exitosa.

Se identificó la necesidad técnica de personalizar la interfaz del Dashboard. Para ello, se expandió la captura de información durante el proceso de autenticación inicial. Se incorporó la asignación de \$_SESSION['usuario_nombre'] dentro de la lógica de iniciarLogin(), vinculándola directamente con los registros de la base de datos. Gestión de Persistencia de Sesión: Cualquier modificación estructural en el manejo de sesiones requiere la invalidación de los tokens activos. Es imperativo ejecutar un logout para forzar una nueva carga de variables en la memoria de PHP.

13. Refactorización del Controlador (Clean Code) y Arreglos Asociativos Objetivo: Mantener el AdminControlador limpio y estructurar los datos que se envían a la vista.

Funciones Privadas: En lugar de tener una sola función gigante que haga todo, creamos funciones de apoyo privadas (private function obtenerUsuarios()). Estas funciones solo pueden ser usadas por el propio controlador y ayudan a que el código sea modular, fácil de leer y de mantener. El Arreglo \$datos: En el patrón MVC, es una excelente práctica agrupar toda la información que el controlador extrajo de la base de datos dentro de un único arreglo (ej: \$datos = ['totalUsuarios' => X, 'totalTorneos' => Y]). De esta forma, le entregamos a la Vista un solo "paquete" con todo lo que necesita imprimir.

14. Modelo: Torneo.php Objetivo: Extender la capacidad de nuestro sistema para interactuar con la tabla de torneos en MySQL.

Estandarización: Al igual que el modelo Usuario, le aplicamos el patrón Singleton en el constructor (Conexion::getInstance()->getBD()) para asegurar el rendimiento. Filtros SQL: Implementamos el método obtenerTodos() usando la instrucción COUNT(*) acompañada de una cláusula WHERE para contar exclusivamente los torneos que están en estado activo o de inscripciones abiertas, descartando los cancelados o finalizados.

15. Arquitectura de Vistas (Plantillas Anidadas) Objetivo: Reemplazar el viejo enrutador falso de Javascript por inyección directa de PHP.

Separación Cascarón / Contenido: Mantenemos la estructura profesional de las maquetas donde dashboard.php funciona como el cascarón (contiene el , los estilos, el menú lateral y el pie de página). Inyección Dinámica (require_once): Dentro del cascarón, utilizamos PHP para inyectar la vista interna (inicio.php que contiene las tarjetas) usando require_once **DIR** . '/inicio.php';. Esto garantiza que PHP arme el rompecabezas completo en el servidor antes de enviárselo al navegador, manteniendo la seguridad de la sesión intacta.

16. Integración de Tablas Relacionales (SQL Avanzado) Objetivo: Obtener métricas complejas cruzando información de múltiples tablas en el Modelo Equipo.php.

Uso de LEFT JOIN y GROUP BY: Para poder contar cuántos miembros tiene un equipo, abandonamos las consultas simples de una sola tabla. Utilizamos un LEFT JOIN para fusionar la tabla equipos con equipo_miembros, y luego agrupamos (GROUP BY) los resultados por ID de equipo. Finalmente, la función COUNT(em.usuario_id) se encargó de calcular el total de integrantes por fila de manera dinámica y altamente eficiente.

17. Inyección de Datos Dinámicos en Vistas (Bucles foreach) Objetivo: Reemplazar el código HTML estático de las tablas del Dashboard por filas generadas dinámicamente según los registros de la base de datos.

Procesamiento de Arreglos: El Controlador recupera los registros utilizando el método fetchAll(PDO::FETCH_ASSOC), obteniendo un arreglo multidimensional. Renderizado con foreach: En la vista (inicio.php), se emplea la estructura de control foreach para iterar sobre el arreglo. Esta técnica instruye a PHP para generar una fila HTML () por cada registro iterado. Manejo de Estados Vacíos: Se implementa la validación empty() para asegurar que, ante la ausencia de registros en la base de datos, la interfaz mantenga su integridad estructural y presente un mensaje informativo al usuario.

18. Control de Acceso Basado en Roles (RBAC) y Seguridad Objetivo: Restringir el acceso a rutas administrativas, garantizando que usuarios con privilegios inferiores (Ej: Jugadores) no puedan ingresar a módulos críticos.

Implementación en la Clase de Seguridad (Sesion.php): Se desarrolló el método estático validarAdmin(), el cual ejecuta una validación de autorización estricta. Se verifica que el rol_id en la variable superglobal \$_SESSION corresponda al valor 1 (Administrador General). De no cumplirse esta condición, el sistema interrumpe la carga de la página mediante exit y redirige la petición hacia el módulo de acceso con un parámetro de denegación. Aplicación en el Controlador (AdminController.php): El método constructor (__construct) fue actualizado para invocar Sesion::validarAdmin() previo a cualquier instanciación de modelos o renderizado de vistas, garantizando el blindaje automático de la clase.

19. Enrutamiento MVC desde la Interfaz de Usuario Objetivo: Interconectar el menú lateral con el Front Controller (index.php) para habilitar la navegación dinámica entre módulos bajo el patrón MVC.

Refactorización de Enlaces: Se sustituyeron los anclajes HTML estáticos por rutas absolutas estructuradas (ej: `href="index.php?c=usuario&a=index"`). Esta nomenclatura instruye al enrutador central para instanciar el Controlador pertinente y ejecutar la acción requerida. **Habilitación de Controladores:** Se reacondicionó el archivo `UsuarioControlador.php`, implementando su respectivo constructor con validación de roles y un método `index()` como punto de entrada para la gestión de usuarios.

20. **Arquitectura de Interfaz y Experiencia de Usuario (UX)** **Objetivo:** Alinear la estructura de navegación del Administrador General con los requisitos funcionales documentados en las especificaciones del proyecto.

Reestructuración Lógica del Menú: Se diseñó una nueva jerarquía de navegación basada en tres bloques funcionales:

1. **Gestión Humana:** Centraliza la administración de todas las entidades (Administrativos, Organizadores, Jugadores y Equipos).
2. **Gestión de Competencias:** Agrupa las facultades operativas de nivel superior (Catálogo de Juegos, Supervisión de Torneos, Registro Global de Resultados).
3. **Configuración del Sistema:** Aísla las funciones de mantenimiento y auditoría (Roles, Alertas Globales, Logs). **Justificación de Diseño:** Esta categorización optimiza la usabilidad (UX) del software, disminuye la carga cognitiva durante la navegación y establece bases para una programación modular de futuros Controladores.
4. **Patrón de Plantilla Maestra (Inyección Dinámica de Vistas)** **Objetivo:** Mitigar la duplicación de código HTML estructural (cabeceras, menús y pies de página) mediante el uso de un contenedor maestro reutilizable.

Resolución Arquitectónica: Se refactorizó el archivo `dashboard.php` para cargar vistas internas de forma dinámica a través de la variable `$vistaInyectada` (ej: `require_once DIR . '/' . $vistaInyectada;`). **Responsabilidad del Controlador:** El Controlador asume la responsabilidad de definir el valor de `$vistaInyectada` previo a la invocación de la plantilla. Esta modificación permite un ensamblaje dinámico y centralizado de las vistas en el servidor.

22. **Pantalla de Gestión de Usuarios (`usuarios_index.php`)** **Objetivo:** Implementar una interfaz administrativa que exponga el directorio completo de cuentas registradas en el sistema.

Modelo de Datos (INNER JOIN): Se desarrolló el método `obtenerTodosUsuariosConRoles()` en la clase `Usuario`. La consulta implementa un INNER JOIN con la tabla `roles`, permitiendo vincular la llave foránea (`rol_id`) con la descripción nominal del rol, optimizando así la lectura de los datos. **Vista y Lógica Condicional:** Se integró lógica de presentación dentro de las iteraciones de la tabla. Mediante estructuras condicionales en PHP, la interfaz renderiza componentes visuales diferenciados (etiquetas de "Activo" o "Bloqueado") basándose en el estado booleano almacenado en la base de datos.

23. **Patrón de Diseño: Strategy (Arquitectura de Algoritmos)** **Objetivo:** Encapsular algoritmos complejos y variables (como la generación de fixtures y cruces) fuera de la clase principal, permitiendo su intercambio dinámico sin alterar el núcleo del sistema.

Planteamiento del Problema: Si la clase `Torneo` debiera gestionar internamente la generación de fixtures para modalidades de Liga, Eliminación Directa y Suizo, se generaría un bloque de código monolítico e insostenible

(violación del principio Open/Closed de SOLID). Solución Arquitectónica (Strategy): Se delegó la responsabilidad algorítmica a clases independientes (ModuloLiga, ModuloEliminacion, ModuloSuizo). La clase Torneo interactúa con estas clases como "estrategias" conectables. Beneficio Operativo: Alta escalabilidad. La integración de futuros formatos de competición (ej. Doble Eliminación o Fase de Grupos) solo requiere el desarrollo de una nueva clase módulo, preservando la integridad y estabilidad del código base.

24. Patrón de Diseño: Singleton (Gestión de Instancias) Objetivo: Garantizar que durante todo el ciclo de ejecución de la aplicación, exista una única instancia global para la conexión a la base de datos, previniendo el agotamiento de recursos del servidor (Too Many Connections).

Implementación en Conexion.php: Se privatizó el método constructor (`__construct()`) para bloquear la instanciación externa (`new Conexion()`). Controlador de Instancia: Se centralizó el acceso a través del método estático `getInstance()`. Este método verifica la existencia previa de la conexión; si no existe, la crea, y si ya existe, retorna la referencia activa. Beneficio Operativo: Optimización de recursos y mitigación del riesgo de colapso de MySQL ante solicitudes simultáneas concurrentes.

25. Centralización de Controladores y Redirección Dinámica (Gestión de Usuarios) Objetivo: Evitar la duplicación de código en operaciones comunes (editar perfiles, cambiar estado) manteniendo interfaces especializadas por rol.

Patrón de Controlador Multipropósito: Se refactorizó `UsuarioControlador.php` para actuar como manejador central de operaciones CRUD. Aunque las vistas y listados estén divididos por sub-controladores (`JugadorControlador`, `OrganizadorControlador`), todas las acciones de guardado y edición convergen en un único punto. Redirección Dinámica basada en Roles: Para mantener un flujo de UX transparente, el método `actualizarUsuario()` inspecciona la variable `$datosActualizar['rol_id']` y utiliza esta información para redirigir (con header) al administrador de vuelta a la tabla específica (`Jugadores` u `Organizadores`) desde la cual se origina la petición.

26. Suplantación de Identidad (Impersonation) Objetivo: Otorgar al Administrador la capacidad temporal de operar bajo los permisos y la vista de otro usuario registrado para facilitar soporte técnico y auditoría.

Gestión Avanzada de Sesiones: Se diseñó el método `verComo()` dentro de `UsuarioControlador`. Este método respalda el ID original del administrador en `$_SESSION['admin_antes_suplantar']` antes de sobrescribir las variables críticas de sesión con los datos del usuario objetivo. Responsabilidad Arquitectónica: Al ubicar esta función en `UsuarioControlador` en lugar de un sub-controlador específico, se respeta el Principio de Responsabilidad Única (SRP), permitiendo su reutilización universal para cualquier rol del sistema sin generar acoplamiento.

27. Patrón Flash Messages (Notificaciones Toast Híbridas) Objetivo: Proveer retroalimentación visual no bloqueante (Toast Notifications) al usuario tras completar operaciones de backend, prescindiendo de AJAX.

Interconexión PHP-Javascript: Se implementó un flujo híbrido. El Controlador define `$_SESSION['mensaje']` y ordena una redirección. La vista maestra (`dashboard.php`) actúa como listener: si detecta la variable en memoria, inyecta dinámicamente un bloque `<script>` que dispara la función frontend `mostrarNotificacionToast()`, procediendo inmediatamente a destruir la variable (`unset`) para garantizar que el mensaje sea efímero (Flash Message).